



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 09

NOMBRE COMPLETO: ALFONSO D'HERNÁN RODRÍGUEZ
ZULUAGA

N° de Cuenta: 321561556

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 05

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 28 de Abril del 2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa.

Hacer que el número sea visible.

Código

```
//número cambiante
/*
¿Cómo hacer para que sea a una velocidad visible?
*/
timer_texture += 1;
if (timer_texture % 60 == 0) {
    toffsetnumerocambiau += 0.25;
}
if (toffsetnumerocambiau > 1.0)
    toffsetnumerocambiau = 0.0;
```

Ejecución





2.- Giro de la aeolipile.

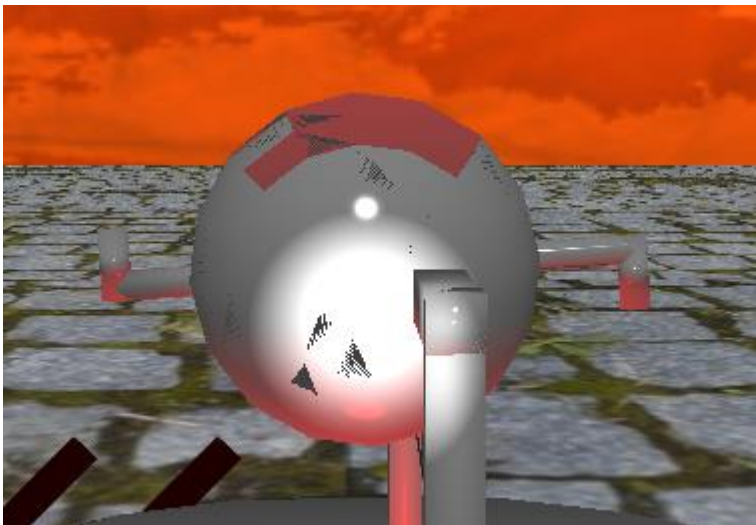
Código

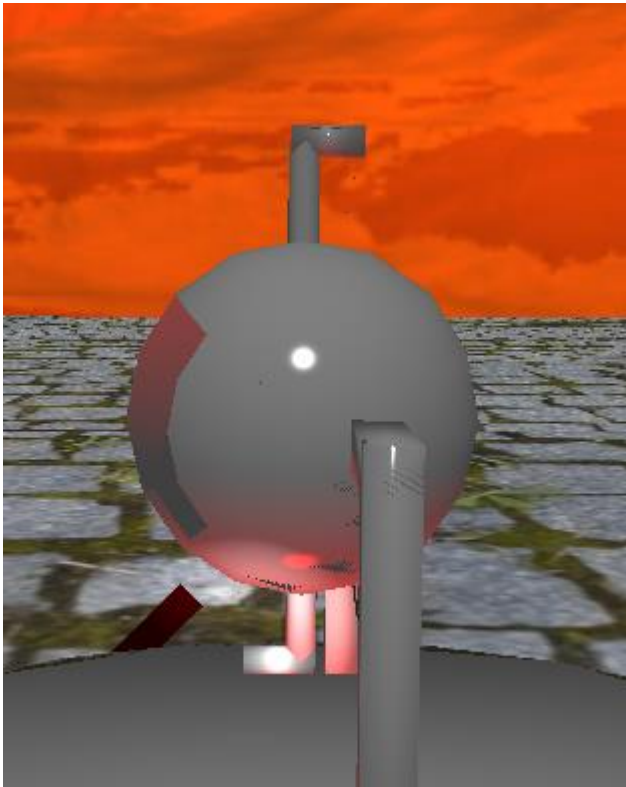
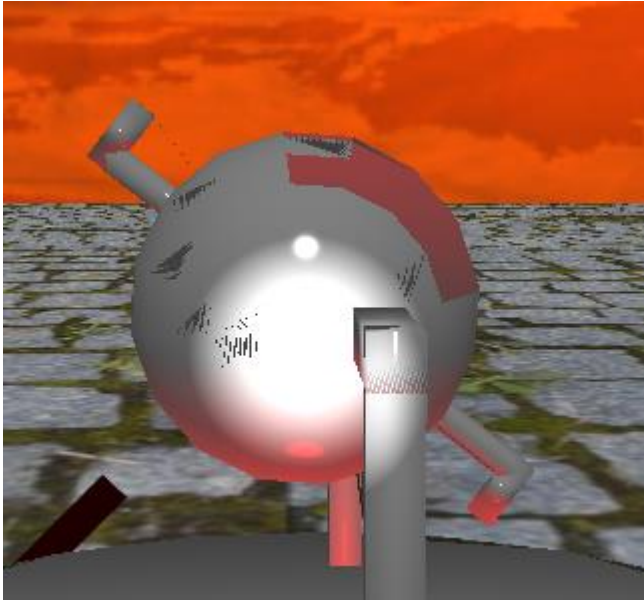
```
//AEOLIPILE
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Aeolipile_base_M.RenderModel();

model = glm::translate(model, glm::vec3(0.0f, 5.0f, 0.0f));
model = glm::rotate(model, aeolipile_rotation * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Aeolipile_M.RenderModel();

if (mainWindow.getPrendeFuego() == 1) {
    if (timer_fire == 0) {
        lugar_fuego += 4.0f;
    }
    aeolipile_rotation += 0.5f;
    toffsetfuego -= 0.01f;
    timer_fire += 1;
    if (toffsetfuego == 1.0f) {
        toffsetfuego = 0.0f;
    }
    if (timer_fire == 3600) {
        mainWindow.setPrendeFuego(0);
        timer_fire = 0;
        lugar_fuego -= 4.0f;
    }
}
```

Ejecución





3.- Hacer que el fuego se prenda.

Código

Window.h

```
int prende_fuego = 0;
```

```
int getPrendeFuego() { return prende_fuego; }
```

```
void setPrendeFuego(int value) { prende_fuego = value; }
```

Window.cpp

```
if (key == GLFW_KEY_F) {  
    theWindow->prende_fuego = 1;  
}
```

Main

```
// fuego  
toffsetnumerou = 0.0;  
toffsetnumerov = 0.0;  
toffset = glm::vec2(toffsetnumerou, toffsetfuego);  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(0.0f, -3.0f + lugar_fuego, 1.5f));  
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));  
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));  
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
fuego.UseTexture();  
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);  
meshList[5]->RenderMesh();
```



```

if (mainWindow.getPrendeFuego() == 1) {
    if (timer_fire == 0) {
        lugar_fuego += 4.0f;
    }
    aeolipile_rotation += 0.5f;
    toffsetfuego -= 0.01f;
    timer_fire += 1;
    if (toffsetfuego == 1.0f) {
        toffsetfuego = 0.0f;
    }
    if (timer_fire == 3600) {
        mainWindow.setPrendeFuego(0);
        timer_fire = 0;
        lugar_fuego -= 4.0f;
    }
}

```

Ejecución





2. Problemas que se tuvieron:

- Tuve problemas al entender el funcionamiento de `deltaTime` y `lastTime` para usarlos en la limitante de cuánto tardaba el número cambiante en moverse en el mapa UV. Para resolverlo, solo definí un entero nuevo.
- Tardé un rato en entender cómo funcionaba el `offset` en `glfw2v` para el `offset`, pero habiendo entendido no hubo mayor complicación. Simplemente tuve que ver el código.
- Hubo un pequeño tropezón para definir el final de la animación, pero solo tuve que hacer un `setter`.

3. Conclusión:

- Los ejercicios fueron bastante fáciles de hecho, una subversión de expectativas comparando con lo que esperaba.
- La explicación en clase estuvo muy bien, entendí todo lo expuesto.